

CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A source-code diffing tool needs an accurate, efficient way to map one abstract syntax tree (AST) onto another. Decades of research have given the field exact tree-edit-distance algorithms, Zhang-Shasha and APTED, and source-code-specific heuristics, GumTree, plus a newer generation of practical, tree-sitter-based tools, difftastic and diffsitter. This paper presents CODEDIFF, a syntax-aware code-diffing tool that builds directly on this line of work, combining APTED’s exact tree edit distance, applied to bounded subtree pairs rather than to whole files, with GumTree’s move-recovery heuristic and Myers’ $O()$ sequence diff into one five-phase matching pipeline. CODEDIFF’s own contribution is not a new matching algorithm but two engineering disciplines applied on top of this foundation. First, its performance and robustness targets come from an empirical study of 100 real-world open-source repositories, about 1.35 million files in 30 languages, not from worst-case asymptotic analysis alone. Second, each optional heuristic pass ships only if an ablation study shows a measurable accuracy gain on a corpus of 468 real-world diffs with human-verified ground truth, so that a technique’s published benefit is confirmed in CODEDIFF’s own pipeline rather than assumed to transfer unchanged. On that same ground-truth corpus, CODEDIFF maps 99.94% of AST nodes correctly, and reaches that accuracy at a speed and reliability suited to everyday, interactive use—read alongside GumTree, difftastic, and diffsitter not as a replacement for any of them, but as one way of combining what each already does well.

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference ’26, June 03–05, 2026, Ludbreg, Croatia

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/26/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference ’26)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is exactly the kind of mechanical, attention-taxing work humans do least reliably.

One way to compute diffs with a genuine move operation is to compare the code’s abstract syntax tree (AST) instead of its raw text. An AST diff maps nodes of one tree onto nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption, and we argue the central reason is cost: the best exact tree-diffing algorithms are $O(n^3)$ in tree size, and Section 3 measures directly how often a whole-tree computation at real-world file sizes exceeds an interactive time budget.

This paper presents CODEDIFF, a syntax-aware code-diffing tool that combines established algorithms with rigorous engineering practice to meet explicit performance and robustness targets. The targets come from an empirical study of real open-source projects—the sizes and shapes of their files and

of the changes developers actually make—rather than from worst-case analysis alone.

The paper is organized around three research questions, each answered by the correspondingly numbered research answer, RA1–RA3, set in a box at the point in the paper where the measurement supporting it is reported.

- **RQ1 (Scale).** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget? Answered by RA1, Section 3.
- **RQ2 (Tool accuracy).** How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes? Answered by RA2, Section 5.
- **RQ3 (Shapes and heuristics).** Which shapes of real-world change—renames, moves, restructurings—require a dedicated matching heuristic, and do heuristics shown effective in one pipeline and corpus retain that value when reused in a different pipeline and evaluated against a different corpus? Answered by RA3, Section 5.

This paper makes four contributions, one empirical, one system, one dataset, and one evaluation:

- **Empirical study.** An empirical study of 100 real-world open-source repositories, showing that the worst-case inputs tree-diffing algorithms are analyzed against are rare in practice, but that the tail is real and large enough that a production tool must handle inputs beyond what a whole-tree tree-edit-distance computation can process within an interactive budget.
- **System.** CODEDIFF itself, a five-phase diffing pipeline built from APTED tree edit distance, scoped to bounded subtree pairs rather than to whole files, together with GumTree’s move-recovery heuristic and Myers’ sequence diff, engineered as one production tool meeting the targets the empirical study sets.
- **Dataset.** A corpus of 468 real-world code changes in 24 languages, each carrying a human-authored ground-truth AST mapping in which every node is labeled with one of seven operations, produced with a purpose-built annotation tool (Section 5). These human solutions are the ground truth every accuracy number in this paper is measured against, and the methodology behind them is reproducible from the project’s research repository.
- **Evaluation.** A quantified comparison of four established diffing tools—GumTree, Unix `diff`, `diffstastic`, and `diffsitter`—against that ground truth, together with a census of which change shapes occur in it and a leave-one-out ablation study of the heuristics targeting them, showing both that a heuristic’s published benefit does not automatically transfer into a different pipeline and that the shape carrying the most remaining error has no dedicated heuristic at all.

2 BACKGROUND

This section introduces the algorithms and tools the rest of the paper depends on: the algorithms CODEDIFF builds its pipeline from, and the tools it is evaluated alongside. Section 6 returns to the closest prior work in more depth once the paper’s own results are in hand.

2.1 Algorithms

Zhang-Shasha [9] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later algorithms reduce this overhead while keeping the same exact result.

APTED [8] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require.

Myers’ O(ND) algorithm [7] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when two sequences are similar, exactly the common case for a version-controlled file.

2.2 Tools

diffstastic [5] and **diffsitter** [3] are a newer generation of practical, tree-sitter-based diffing tools, the same parsing layer CODEDIFF itself is built on. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: `diffstastic` reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and `diffsitter` applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars. Neither computes a full tree-edit-distance mapping the way APTED or GumTree do, so neither directly targets the ground-truth-accuracy metric

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

this paper measures against in Section 5—a difference in design goal, not a shortcoming, and the reason we include both as companions in the same problem space rather than as head-to-head competitors on identical terms.

3 PROBLEM SPACE

Formal treatments of the algorithms in Section 2 report worst-case asymptotic complexity: APTED and Zhang-Shasha are both $O(n^3)$ for general trees, where n is tree size. This analysis is necessary, but it is not sufficient for a production tool. GumTree’s own documentation states that it targets research use, not production deployment at scale—a reasonable scope for its own goals, but one that leaves open a question every production diff tool must answer:

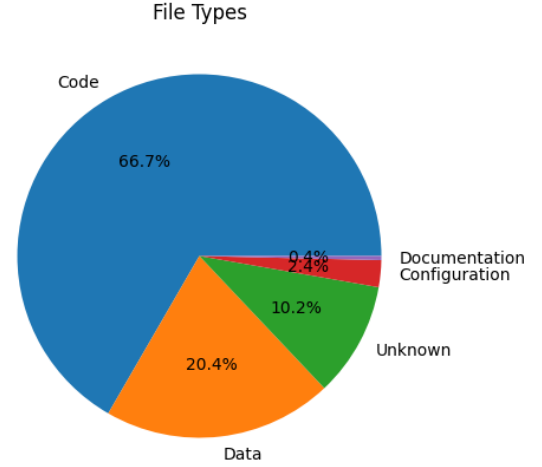
RQ1 (Scale), stated in Section 1 and restated here: what percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget?

A tool engineered only against a worst-case bound risks two failure modes. It can over-engineer for inputs that never occur in practice. It can under-engineer for a real tail its asymptotic analysis never quantified. Answering RQ1 needs measurement, not further asymptotic analysis.

We measure this directly. We built a corpus of 100 open-source repositories, drawn from the Gentoo Linux distribution’s package list and popular repositories on GitHub. We cloned each repository to a depth of 50 commits from every branch tip and parsed every file with Tree-sitter, recording byte size, line count, language, and AST node count per file. The depth limit matters for what follows: it bounds how far back the commit sampling in this section can reach, and a sampled (repository, commit, path) triple stays resolvable only while that commit remains within the depth window of the local clone. The full methodology, and every per-language number, is reproducible from the project’s research repository; here, we summarize the numbers that shape CODEDIFF’s design.

The corpus has 1,347,490 files across 30 languages. About two-thirds of all files are code. The rest is configuration, data, and documentation (Figure 1). Restricted to code files, Table 1 reports size percentiles in bytes, lines of code, and AST nodes.

The largest file in the corpus has 573,334 AST nodes. Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates. Arafat and Riehle report that 47.5% of all commits

**Figure 1: Distribution of file types across the corpus.**

touch 10 lines or fewer [1], while commits of over 10,000 lines still occur.

Measurement. To answer RQ1’s percentage directly, we sampled 2,922 real (before, after) file pairs from single-parent commits in these repositories, stratified per language and per size bucket so that large files are represented rather than drowned out by the far more common small ones. We ran a single whole-tree tree-edit-distance computation on every pair—CODEDIFF’s own APTED implementation, applied to the full before and after trees with none of CODEDIFF’s pipeline heuristics—under a hard one-second budget enforced by terminating the computation’s process. This implementation includes an optimization that makes it faster than a textbook APTED, so the failure rates below are a lower bound on how often a whole-tree computation misses the budget, not an upper bound.

Every sampled pair was checked to still resolve against the local clones before measurement, and all of them did. This is worth stating because the check has failed before: an earlier sample, drawn against a shallower corpus, had decayed to roughly 41% unresolvable by the time it was re-measured, and the losses fell on whole repositories rather than spreading evenly across the sample. A measurement over that sample would have reported a healthy-looking rate while quietly answering a question about a different, smaller corpus.

Figure 2 shows the fraction of pairs completed within one second, per size bucket, split by what kind of artifact the file is: code (1,942 pairs across general-purpose programming languages), configuration, data, and markup formats (700 pairs), and scripting languages (280 pairs). The split matters because the three populations behave differently—blended, the headline number would move with how many configuration files a sample happens to contain, independently of anything the algorithm does to source code.

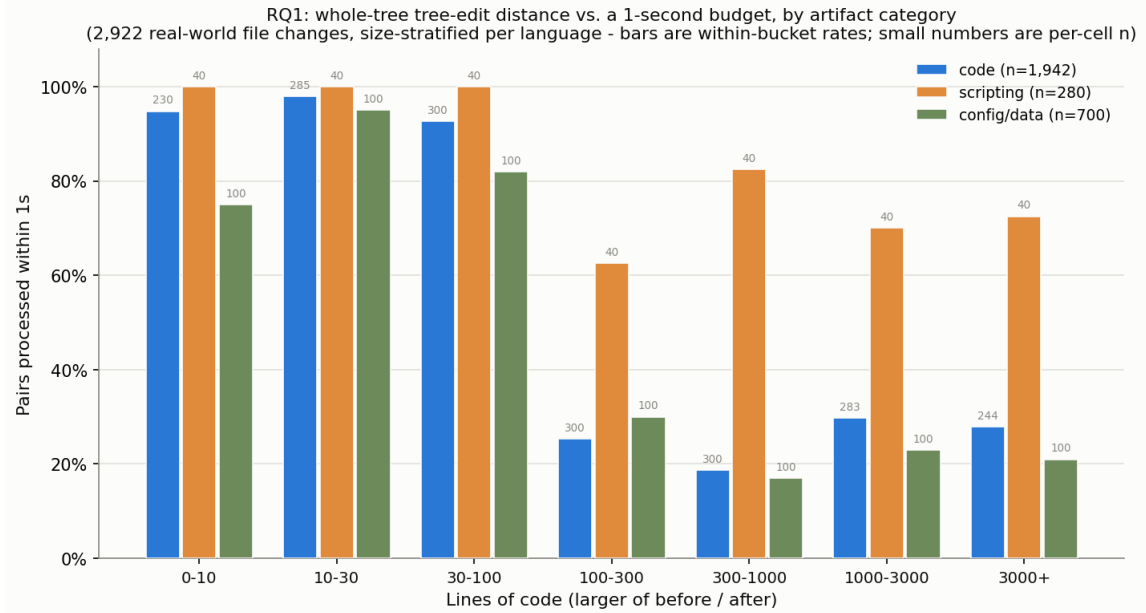


Figure 2: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

RA1 (Scale)

For source code, a whole-tree tree-edit-distance computation is dependable only for small files, and the transition is abrupt rather than gradual: it completes within one second for 97.9% of code pairs of 10–30 lines, but for only 25.3% of code pairs of 100–300 lines, and never recovers above one third in any larger bucket. Across all sampled code pairs, 54.5% complete in time.

What makes that a design problem rather than a tail problem is where the collapse falls: between two adjacent buckets, neither of which is an unusual size for a source file, and both well inside the range ordinary source files occupy (Table 1). The corpus-wide figure is a sample-weighted aggregate: it deliberately over-represents large files and therefore understates the rate a uniform sample of real commits would show, but the per-bucket rates carry the answer regardless of weighting.

The three artifact categories do not behave alike. Scripting languages complete within the budget 83.9% of the time, well clear of code at 54.5% and configuration, data, and markup formats at 49.0%; the latter two are close to each other and separated by less than six points, so the meaningful split is scripting against the rest rather than a clean three-way ordering. Figure 2 shows the same pattern per bucket: scripting stays above every other category in all seven, while configuration and data fall furthest in the large buckets, despite containing the most repetitive and most regularly structured files in the corpus. We conjecture this reflects tree edit distance being sensitive to tree shape rather than to node count alone, a large flat sequence of similar siblings being an

unfavorable shape for it—the same reasoning CODEDIFF’s pipeline acts on in Section 4, where large flat subtrees are pre-matched with a sequence algorithm rather than handed to tree edit distance at all. This measurement does not isolate shape from size, so we state it as a conjecture rather than a result.

The common case is genuinely cheap—most files and most commits are small—but the tail is real: files up to 573,334 AST nodes occur in this corpus, and Arafat and Riehle’s data shows commits of over 10,000 lines occur in practice. A production diff tool must be engineered for both regimes, and cannot rely on a whole-tree computation alone for either. Section 4 turns this answer into CODEDIFF’s concrete design targets.

4 CODEDIFF

The measurements in Section 3 give CODEDIFF two concrete, named design targets, stated in place of an asymptotic bound. **Robust:** CODEDIFF must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast:** CODEDIFF must compute a diff in under 400 ms for 99.99% of real-world commits, the range that covers the overwhelming majority of commits per Arafat and Riehle’s measurement. Section 5 reports how well CODEDIFF meets both targets in practice.

CODEDIFF matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. CODEDIFF matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. Each variant claims what it can before the next runs. This phase is CODEDIFF’s own design, motivated directly by Section 3’s commit-size data: since most commits touch a small fraction of a file’s lines, most of a typical file’s AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation’s cost on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below. Neither is reachable by phase 1’s structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it.

Phase 3, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run, large flat subtrees, such as a macro’s token list or a big flat argument list, are pre-matched directly with Myers’ algorithm [7], since a flat sequence has no tree structure worth exploiting—exactly the structures that drive Section 3’s measured AST-size tail. Each accepted pair is then handed to APTED [8], scoped to that pair alone, to compute its exact internal edit script. This is the only place a tree-edit-distance computation runs at all, and the scoping is what keeps it affordable: APTED sees one accepted container pair at a time, never the whole file.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [7] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual the alignment must guess at, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Earlier versions of CODEDIFF instead ran one whole-residual APTED computation here, with the Myers-LCS alignment substituting only when that computation looked too expensive. That design was abandoned: its cost is driven by residual *shape* rather than size, which makes it impossible to gate reliably. Two measured cases make the point—an 11,647-node Vimscript residual took 87 seconds, while a 258,504-node JSON residual took 9.4. No size threshold separates those

two, so the exact computation was removed from the whole-file path entirely and survives only in phase 3’s scoped form. This is a direct consequence of the size tail in Section 3: an unbounded whole-residual computation cannot meet the 400 ms target at any threshold.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree’s recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. It runs last by necessity rather than by preference: every phase above requires some anchor—a hash, a name, a shared matched ancestor—before it will consider a pair at all, and content that moved between two containers that both changed identity has none. Section 5 finds this pass contributes more accuracy than every other optional pass in the pipeline combined.

Zhang-Shasha [9] plays no role in this pipeline as a shipped algorithm. CODEDIFF’s own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED’s result always matches Zhang-Shasha’s, bit for bit. This checks the pipeline’s most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

5 EVALUATION

Dataset. CODEDIFF’s accuracy claims rest on a corpus of 468 fixtures spanning 24 languages. Each fixture pairs a before and after version of a source file, representing a realistic code change—a bug fix, a refactor, an added feature, or a performance-motivated rewrite. Most fixtures are captured directly from a specific commit in a real open-source repository; the rest are curated examples of a common, realistic change pattern. For every fixture, a human annotator used a purpose-built tool, `human_solver`, to walk the before and after syntax trees side by side and label every node with one of seven operations: identical, update, match-but-not-identical, insert, insert-with-children, delete, and delete-with-children. Nodes are addressed by a path of kind and sibling position, not by parser-internal node identity, since that identity is not stable across separate parses of the same file. This human-authored mapping is the ground truth every number in this section is measured against.

This section answers RQ2 and RQ3, stated in Section 1 and restated below, building on RA1’s finding that a real, non-negligible tail of large files and commits exists alongside a small common case, and then evaluates CODEDIFF itself—the paper’s system contribution—against the design targets stated in Section 4.

RQ2 (Tool accuracy). How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes?

RQ3 (Shapes and heuristics). Which shapes of real-world change—renames, moves, restructurings—require a dedicated matching heuristic, and do heuristics shown effective in one pipeline and corpus retain that value when reused in a different pipeline and evaluated against a different corpus?

Comparison tools. We place CODEDIFF alongside four established tools, on the same 468-fixture corpus: GumTree (v4.0.0-beta8) [4], Unix **diff**, and the two tree-sitter-based tools from Section 2, difftastic [5] and diffsitter [3].

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. difftastic and diffsitter deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree remains the foundation CODEDIFF’s move-recovery phase builds on directly (Section 4), and its broader language and backend flexibility stays valuable well beyond what this single corpus measures. One consequence of that flexibility is visible in the numbers below: every generator GumTree offers is included here, and all but two of them are marked “Testing” rather than “Stable” by GumTree’s own documentation, so its rate reflects a deliberately wide language scope rather than its performance on the backends it considers ready. Read what follows as a report of where combining these approaches into one production-engineered, ablation-validated pipeline lands, not as a ranking of which tool is best.

AST-node accuracy. Measured directly against the human mapping, CODEDIFF maps 4,956,544 of 4,959,531 AST nodes correctly across the 468 fixtures, 2,987 mismatches, or 99.94% node-level accuracy. That denominator counts every AST node, including every ancestor of every change up to the root, so it partly reflects how deeply a given grammar nests. Restricted to the nodes that carry text of their own and therefore reach the screen when the diff is rendered, the figure is 99.94%, 2,037 mismatches out of 3,380,690. The two rates agreeing to two decimal places is itself the useful result: CODEDIFF’s residual errors are not concentrated in invisible structural nodes, so the headline figure is not being flattered by nodes no reviewer ever sees.

Line-level agreement. None of the other four tools report an AST-node mapping, so we compare all five on a common ground: which source lines each tool marks as touched, against the same human mapping (Table 2, Figure 3). Coverage varies widely: diffsitter’s compiled-in grammar set matches 292 of the 468 fixtures’ languages, GumTree’s generator set matches 359, difftastic’s 449, and only CODEDIFF and Unix **diff** cover all 468. Each tool’s own row is scored only on its own applicable subset, not padded with unscored fixtures. Because those subsets differ so much, the table also reports every tool on the 250 fixtures all five scored, which holds the fixture set fixed at the cost of over-representing the mainstream languages every tool supports.

Table 2: Line-level mismatch rate against human ground truth, on each tool’s own applicable subset of the 468-fixture corpus and on the 250 fixtures every tool scored.

Tool	Fixtures	Mismatched lines	Rate	Common
CODEDIFF	468	3,491	0.608%	0.882%
Unix diff	468	6,578	1.145%	0.792%
diffsitter	292	5,998	1.421%	1.032%
difftastic	449	9,768	1.819%	1.929%
GumTree	359	15,851	4.409%	5.074%

RA2 (Tool accuracy)

Among the four pre-existing tools (Table 2, excluding CODEDIFF’s own row), line-level mismatch rates against the human mapping range from 1.145% (Unix **diff**) to 4.409% (GumTree). Every tool agrees with the human mapping on the great majority of lines, so what separates them is the small, structurally interesting remainder—and on that remainder no AST-aware tool among the four beats plain line-based **diff**, on either basis. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it.

Two details behind that answer are worth stating. The first is how wide the agreement is: even the highest mismatch rate leaves more than 95% of lines in agreement, so the question that separates tools is a narrow one. The second is where a line-based tool’s advantage comes from—the gap is in which heuristics act on the syntax tree, which RQ3 turns to next.

The two bases also disagree about CODEDIFF itself, and the disagreement is worth stating plainly. Across the whole corpus CODEDIFF has the lowest rate of the five, 0.608% against Unix **diff**’s 1.145%. On the 250-fixture common subset that ordering reverses: Unix **diff** scores 0.792% and CODEDIFF 0.882%. We conjecture the difference is which languages each basis weights—the common subset is exactly the set of mainstream languages with mature grammars, where a line-based diff is hardest to beat, while the fixtures only CODEDIFF and Unix **diff** can both attempt include the markup and configuration formats where structural matching pays off most. We report both rather than choosing the flattering one.

Change-shape census. RQ3 has two halves, and the ablation study below answers only the second. Before asking which heuristics pay for themselves, we measure which shapes of change actually occur. The human-authored mappings record this directly: a mapped node whose path depth differs between the two sides is a reparent, a pair of same-kind siblings that swap order under an unchanged parent is a reorder, and a mapping entry the annotator deliberately left pointing at several candidates is a case where the ground truth itself is ambiguous. Table 3 reports how often each

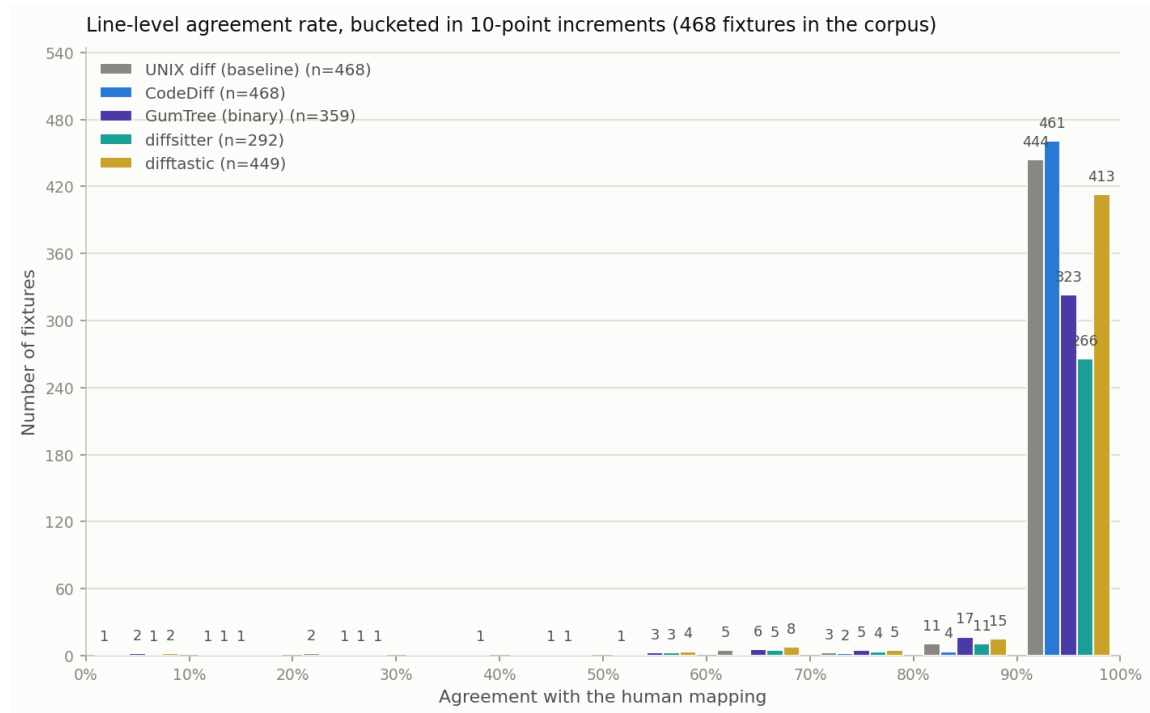


Figure 3: Line-level agreement rate with the human mapping, bucketed in 10-point increments, all five tools on the same axis. Each series’ own n reflects its applicable subset of the corpus.

Table 3: Change shapes in the ground-truth corpus: how many fixtures contain each, and the fraction of those CodeDiff maps with zero mismatches. Rows overlap; the last two are the complement and the whole corpus.

Shape	Fixtures	Share	Zero mismatches
Reparent (depth ± 1)	83	17.7%	28.9%
Reparent (depth ≥ 2)	77	16.5%	28.6%
Sibling reorder	12	2.6%	41.7%
Ambiguous ground truth	47	10.0%	19.1%
None of the above	328	70.1%	90.9%
Whole corpus	468	100%	73.9%

occurs across the 468 fixtures, and how often CODEDIFF maps a fixture containing it with no mismatches at all.

The shapes are not mutually exclusive and one fixture can contain several, so the rows do not sum to the corpus. The reorder row in particular is a floor rather than a census: it counts only same-kind siblings directly under an unchanged parent, so a branch swap that also changes wrapper depth is counted as a reparent instead.

The census draws a sharp line. 70.1% of fixtures contain none of these shapes, and CODEDIFF maps 90.9% of those perfectly. Every shape category falls far below that: a fixture containing a reparent is mapped perfectly 28.9% of the time,

one containing an ambiguous mapping 19.1%. The corpus-wide 73.9% therefore averages two populations that behave nothing alike.

Weighted by mismatches rather than by fixtures, reparenting at any depth accounts for 58.5% of the corpus’s total error while occurring in 26.7% of its fixtures, and is present in 68.9% of every fixture CODEDIFF does not map perfectly. It is the largest single identified source of error, but not the only one: 35.3% of the remaining mismatches sit in fixtures exhibiting none of these four shapes, so this census names the dominant factor rather than a complete explanation.

Ablation study. RQ3 asks which shapes of change earn a dedicated heuristic. CODEDIFF has four optional heuristic passes, each targeting a specific shape and each behind its own flag: move-detection recovery (whole subtrees relocated across a matched boundary, phase 5), bottom-up propagation (a container whose children all match one container on the other side, phase 4), mutual-ancestor pairing (a container sitting above an edit whose descendants all survive inside one corresponding container), and unique type matching (a matched parent’s single remaining child of a given kind on each side), the last adapted from GumTree Simple’s recovery sub-phase. A leave-one-out ablation study measured each pass’s real contribution on the 468-fixture corpus above, by disabling one pass at a time and comparing the resulting mismatch count against a baseline with all four enabled.

Table 4 reports the result at two granularities: every AST node, and only those nodes that carry text of their own and

Table 4: Ablation study: accuracy change from disabling each optional pass, against a baseline with all four enabled. A positive number means disabling the pass hurt accuracy, more mismatches. Visible nodes are those carrying text of their own, the ones a rendered diff displays. The two passes adapted from GumTree are marked (G).

Pass	Δ all nodes	Δ visible
Move-detection recovery (G)	+2,266	+1,564
Bottom-up propagation	+318	+0
Mutual-ancestor pairing	+23	+0
Unique type matching (G)	+0	+0

therefore reach the screen when the diff is rendered. The two disagree in a way that matters. Across all nodes, three of the four passes earn their place; restricted to what a reviewer actually sees, only one does. Bottom-up propagation’s +318 nodes and mutual-ancestor pairing’s +23 are, without exception, structural interior nodes—the containers and wrappers a rendered diff never displays on their own. Unique type matching moves neither column.

RA3 (Shapes and heuristics)

On this corpus the move is the one shape whose dedicated heuristic pays for itself in a way a reader would notice: move-detection recovery contributes more accuracy than the other three optional passes combined, by a factor of seven across all nodes, and is the only one of the four that changes the visible-node count at all. A published benefit does not transfer automatically: unique type matching, GumTree Simple’s own recovery sub-phase, is correct in isolation yet contributes exactly nothing here, because earlier phases reach its cases first. The shape carrying the most remaining error has no dedicated heuristic at all—reparenting occurs in 26.7% of fixtures and carries 58.5% of the corpus’s remaining error.

That the move earns its pass matches the structural argument of Section 4—a relocated subtree is the one shape ordered tree edit distance cannot express except as a delete plus an insert, so no downstream computation can recover it without a dedicated pass.

Unique type matching is the sharper result for RQ3’s second half, and the reason it contributes nothing is not that it is unsound: instrumentation shows it fires *zero* times across the entire corpus. By the time it runs, CODEDIFF’s earlier phases have already resolved every case it targets, and a matched parent’s leftover unmatched children essentially never land on the “exactly one of this kind on each side” configuration it requires. A heuristic’s value is therefore a property of the pipeline it is placed in, not of the technique alone: the same pass that recovers real mappings where it originated can be dead code in a pipeline whose earlier phases are comprehensive enough to reach its cases first. We argue

this is the more transferable lesson, and it is only visible by measuring firings and accuracy inside the adopting pipeline, rather than inheriting the original paper’s result.

Read together with the census, RQ3’s two halves answer each other, and they do not name the same shape. The ablation says which shipped heuristic earns its place: move-detection recovery, which handles relocation across a matched boundary. The census says which shape is still unserved: reparenting—a node that stays where it is but gains or loses an enclosing level, the shape produced by wrapping a block in a condition or a try—has no dedicated pass at all. This is what an ablation cannot report on its own: it can only report on heuristics somebody wrote, and is silent about the shape nobody wrote one for. We argue a shape census belongs alongside an ablation for that reason, not as a substitute for it.

Production viability. The remaining measurements evaluate CODEDIFF itself—speed, then robustness—against the Fast and Robust targets stated in Section 4.

Speed. Table 5 and Figure 4 report wall-clock percentiles over the same fixtures, sorted fastest to slowest by median. We report GumTree twice: once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree’s own matching cost. CODEDIFF is the fastest AST-aware tool at the median (5.9 ms), but this ordering does not hold across the distribution: diffstiter, whose narrower hand-picked grammar set does markedly less work, overtakes it at the 90th percentile and is an order of magnitude ahead at the 99th (239.5 ms against 2037.5 ms). CODEDIFF’s tail is heavy, and Section 4’s design explains why: the phases that resolve the common case cheaply do nothing for a large, genuinely dissimilar residual, which is exactly what the slowest fixtures in this corpus are. diffstastic’s tail is heavier still in aggregate cost and essentially ties CODEDIFF at the 99th percentile. Unix `diff` remains fastest at every percentile, since it never parses or compares tree structure.

The two GumTree rows bracket how much of its cost is JVM startup rather than matching. A warm JVM takes its median from 510.2 ms to 73.5 ms, so most of the per-invocation figure is process startup, not GumTree’s algorithm. We report both because each answers a different question: the cold number is what a developer running a diff from the command line actually waits for, and the warm number is what GumTree’s matching costs once that is amortized away.

These percentiles must be read against the right population. The Fast target of Section 4—400 ms for 99.99% of real-world commits—is a claim about the real distribution of commits, in which 47.5% touch 10 lines or fewer [1]. The fixture corpus is not that distribution: it deliberately concentrates hard, structurally interesting changes, because easy changes exercise nothing worth measuring. CODEDIFF’s 99th percentile on this corpus (2037.5 ms) is therefore a stress figure on an adversarially weighted sample, not the target metric; its median (5.9 ms) and 90th percentile (140.2 ms) on the same hard sample sit well inside the budget. Validating

Table 5: Wall-clock time percentiles, milliseconds, sorted fastest to slowest by median.

Tool	p50	p90	p99
Unix diff	2.6	3.3	13.9
CODEDIFF	5.9	140.2	2037.5
diffsitter	7.7	59.1	239.5
diffastic	50.5	157.2	2036.7
GumTree (warm JVM)	73.5	748.6	5681.6
GumTree (cold, per-invocation)	510.2	1318.5	6559.3

Table 6: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	<i>n</i>	Median CoV	p90 CoV
TreeSitter parse (lower bound)	468	3.0%	17.1%
UNIX diff (baseline)	468	5.8%	8.2%
CodeDiff	468	2.3%	9.7%
GumTree (binary)	359	3.6%	7.4%
GumTree (warm JVM)	359	3.0%	15.9%
diffsitter	292	2.5%	6.0%
diffastic	449	1.5%	24.8%

the 99.99% claim on a uniformly drawn commit sample remains future work, and we state it here as a design target rather than a measured result.

Table 6 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

Robustness. We sampled 925 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CODEDIFF, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 925 sampled pairs, 377 were unavailable locally because the repository’s history had since been rewritten, not a CODEDIFF failure, and 142 exceeded the node cap and were skipped before CODEDIFF ever ran. CODEDIFF completed all 406 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, and the 142 pairs above the node cap are precisely the inputs the Robust target is about, so they are excluded from the evidence rather than counted in its favor. Within those limits the result is consistent with CODEDIFF’s stated Robust target in Section 4.

Summary. Combining established techniques rather than inventing new ones, CODEDIFF reaches 99.94% AST-node accuracy against human-verified ground truth, the lowest line-level mismatch rate of the five tools compared over the corpus as a whole, and zero panics or timeouts across the

sampled robustness evaluation. Building a production-grade syntax-aware diffing tool did not require a new matching algorithm, only careful combination and measurement of existing ones.

6 RELATED WORK

Tree-diffing tools. GumTree [4] remains the reference point for source-code tree diffing, and CODEDIFF’s move-recovery phase adapts its recovery-mappings heuristic directly (Section 4). The difference is one of goal: GumTree targets research use, with broad language and backend flexibility, while CODEDIFF narrows to a tree-sitter-only frontend and spends the saved generality on production targets—and, as the ablation in Section 5 shows, on validating each adopted heuristic inside its own pipeline rather than assuming the published benefit transfers. diffastic [5] and diffsitter [3] share CODEDIFF’s parsing layer but optimize for a diff a human reads comfortably rather than for fidelity of the underlying node mapping; RA2 reflects that difference in design goal, not a defect in either tool. Unix **diff**, built on Myers’ algorithm [7], remains the baseline every one of these tools must justify itself against—on our corpus it is both the fastest tool measured and, at the line level, more accurate than every pre-existing AST-aware tool compared.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CODEDIFF’s Fast target is stated against. This paper’s empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

7 CONCLUSION AND FUTURE WORK

None of CODEDIFF’s matching algorithms are new. Its contribution is combining APTED’s exact tree edit distance, GumTree’s move-recovery heuristic, and Myers’ sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case, and validating every optional piece of that combination by measurement instead of by assumption. On the corpus in this paper, that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree’s language and backend flexibility, diffastic’s and diffsitter’s speed and readability, Unix **diff**’s universality. The ablation result in Section 5 is the more general, transferable point: a heuristic’s value is a property of the pipeline and corpus it runs in, not only of the paper that introduced it. A production tool must measure that value directly, rather than assume it carries over.

The same discipline applies to the exact algorithm at the pipeline’s core. CODEDIFF began by running one whole-residual APTED computation per file, exactly as its asymptotic analysis invites, and removed it after measurement showed its cost tracks residual shape rather than residual

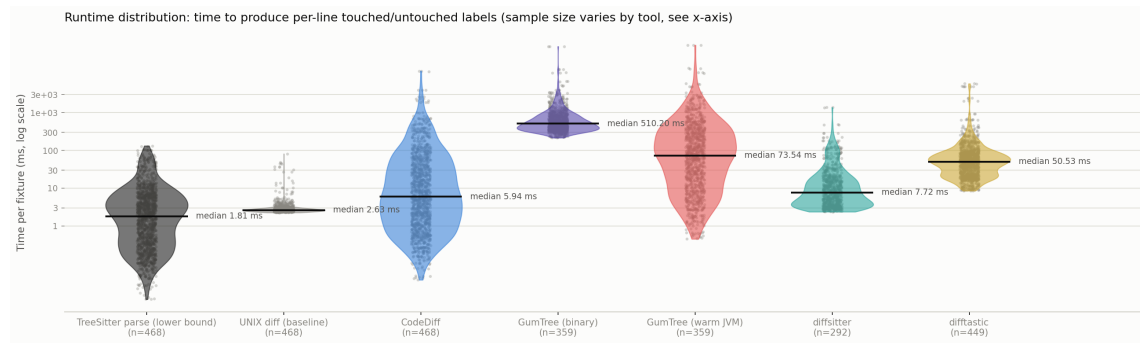


Figure 4: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

size closely enough that no size threshold can gate it safely. What survives is APTED scoped to individual container pairs, where the same exactness is affordable. An exact algorithm is not automatically the right one to run at whole-file scale, and only measurement distinguishes the two regimes.

Based on the lessons reported in this paper, we recommend the following to builders of syntax-aware developer tooling:

- Measure every adopted heuristic inside your own pipeline, against your own corpus, and count how often it fires as well as what it changes: one heuristic adopted here contributes more than all the others combined, while another, correct and unit-tested in isolation, never fires at all.
- Engineer against the measured distribution of real inputs, not against the worst-case bound alone: the common case and the tail require different machinery, and both occur in practice.
- Scope an exact algorithm to the inputs it can serve within budget rather than abandoning or over-applying it: the same computation that is unaffordable over a whole file is the right tool for a bounded subtree pair.
- Keep an exact algorithm as an independent test oracle even when the shipped path is heuristic: CODEDIFF fuzz-checks its APTED implementation against an independently implemented Zhang-Shasha on thousands of random trees.

Future work includes widening the robustness corpus beyond Rust, extending the accuracy comparison to more languages GumTree also supports, and re-running the ablation study as new heuristic passes are added to the pipeline. The clearest direction the evaluation points to is a dedicated pass for the reparenting shape, which carries 58.5% of the corpus's remaining error with no heuristic addressing it: Section 4's phase ordering offers no anchor for a node whose position is unchanged but whose depth is not. The 35.3% of error in fixtures matching none of the censused shapes is a second, less well-characterized target, and suggests the census itself should be extended before more heuristics are written against it.

REFERENCES

- [1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. <https://doi.org/10.1109/HICSS.2009.421>
- [2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. <https://doi.org/10.1145/2568225.2568295>
- [3] Afnan Enayet. 2020. difflitter: A Tree-Sitter-Based AST Diff tool for Meaningful Diffs. <https://github.com/afnanenayet/difflitter>. Accessed 2026-07-31.
- [4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. <https://doi.org/10.1145/2642937.2642982>
- [5] Wilfred Hughes. 2018. DiffTastic: A Structural Diff That Understands Syntax. <https://github.com/Wilfred/difftastic>. Accessed 2026-07-31.
- [6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.
- [7] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. <https://doi.org/10.1007/BF01840446>
- [8] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. <https://doi.org/10.1145/2699485>
- [9] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. <https://doi.org/10.1137/0218082>